

GitHub Branch and Pull Request (PR) Workflow Guide

Purpose

This guide describes a standard GitHub workflow for collaborating in a repository using:

- Branches
- Commits
- Pull Requests (PRs)
- ⚠ Important: Merging into main
- ⚠ Important: Shared Branch for Team Collaboration - How To
- This workflow prevents accidental changes to main and makes collaboration safer.

Basic Concept

- The main branch should always contain stable code.
- Each new task should be developed in its own branch.

```
text
main
├── feature/add-login
├── feature/update-docs
└── bugfix/fix-api
```

- After work is completed:

Branch → Pull Request → Review → Merge into main

Standard Workflow

💡 Step 1: Clone Repository (First Time Only)

```
git clone <https://github.com/Data-Analytics-Study-Group/ravenstack-pl.git>
```

Move into the repository:

```
cd project
```

💡 Step 2: Check Current Branch You Are In

```
git branch
```

This command lists all branches and marks your current branch as `*`.

Example:

```
* main
```

💡 Step 3: Update Local main

Always begin work from an updated main.

```
git checkout main  
git pull origin main
```

What git pull does: Fetches the latest changes from the remote main branch and merges them into your local main.

💡 Step 4: Create a New Branch For Your Work

Please refer to the section 'Branch Naming Convention' below for guidelines on branch naming.

Create and switch to a new branch in one command:

```
git checkout -b prefix/purpose-description
```

Example:

```
git checkout -b data/data-cleaning
```

This command:

- Creates a new branch for your work
- Switches to it immediately

Verify your branch:

```
git branch
```

Example output:

```
* data/data-cleaning
main
```

Branch Naming Convention

Prefix	Purpose
data/	Data cleaning or pipeline changes
explore/	EDA or one-off investigations
pipeline/	ETL/ELT workflows and orchestration (Airflow, dbt, etc.)
fix/	Fixing a bug in code, a broken pipeline, or bad logic
docs/	READMEs, data dictionaries, methodology writeups
eval/	Model evaluation, metrics, and validation work
viz/	Dashboards, charts, and reporting visuals

Rules for the Description

- **2–5 words max** — keep it concise
- **Use imperative form** — `normalize-sales-schema` not `normalized-sales-schema`
- **Use lowercase with hyphens** — `fix-null-join-logic` not `Fix_Null_Join_Logic`
- **Be precise** — anyone reading the branch name should know what it adds to the project
- **Avoid vague names** — `fix/updates` or `data/changes` tell nobody anything

Examples

✓ Good	✗ Bad
data/normalize-sales-schema	data/changes
explore/customer-churn-patterns	explore/analysis
pipeline/daily-revenue-aggregation	pipeline/new-pipeline
fix/null-handling-join-step	fix/bug
viz/executive-kpi-dashboard	viz/dashboard2

! Important: Ensure you are on your branch before making commits to branch.

💡 Step 5: Make Changes

Edit files in your editor or IDE.

!!! IMPORTANT: KEEPING YOUR WORK SAFE WHEN WORKING ON A SHARED FEATURE BRANCH !!!

! Best Practice: Commit (or Stash) Before Pulling

Before running `git pull`, check whether you have any uncommitted changes:

```
git status
```

If your working directory is clean, you can safely pull the latest changes:

```
git pull
```

If you have uncommitted changes, choose one of the following options before pulling.

Option 1 (Recommended): Commit Your Changes

If you have completed a logical piece of work, stage and commit your changes first:

```
git add .  
git commit -m "Describe your changes"
```

Then pull the latest changes:

```
git pull
```

Option 2: Stash Unfinished Work

If your changes are not yet ready to commit, temporarily save them using `git stash`:

```
git stash  
git pull  
git stash pop
```

Why Is This Recommended?

Pulling while you have uncommitted changes can:

- Cause Git to refuse the pull if incoming changes would overwrite your local work.

- Make merge conflicts more difficult to resolve because your changes are not yet saved as a commit.
- Increase the risk of accidentally losing track of unfinished work.

By committing (or stashing) first, your work is safely preserved before incorporating changes from the remote repository.

Best Practice: Pull frequently and commit small, logical units of work. Regular synchronization with the remote repository helps minimize merge conflicts and makes collaboration much easier.

💡 Step 6: Check for Changes to the Branch

Someone else may have committed some work to the branch. Check how your current files differ from the latest commit on the branch you're currently on.

```
git status
```

Example output:

```
modified: script.py
```

💡 Step 7: Stage Files

Stage all modified files `git add .` or stage specific files: `git add <file_name>`:

Example: Stage a specific file

```
git add script.py
```

Or stage all changes (new, modified, deleted) in the current directory:

```
git add .
```

💡 Step 8: Commit Changes

Create a snapshot of your work.

```
git commit -m "Add data cleaning pipeline"
```

Examples of good commit messages:

- Add preprocessing function

- Fix bug in API request
- Update documentation
- Refactor SQL queries

Rule: Always commit on your feature branch, never directly on main.

💡 Step 9: Push Branch & Set Upstream Tracking to GitHub

Specifying upstream tracking is done only for your first push of a new local branch to Github. It tells github which local branch is linked to which remote branch:

- *origin = the GitHub repository*
- *data/data-cleaning (first one) = your local branch*
- *origin/data/data-cleaning (implicitly created) = the remote branch on GitHub*

Push and set upstream tracking (first push):

```
git push -u origin data/data-cleaning
```

Subsequent pushes (after upstream is set, there's no need to keep specifying the branch again):

```
git push
```

Keeping Your Branch Updated

While you work, teammates may merge changes into main. Before opening a Pull Request, you should update your branch with the latest main.

The purpose of **Steps 10–12** is to bring the latest changes from main into your branch, so your branch stays current before opening a PR.

💡 Step 10: Update Local main

To ensure your branch is up to date with **main**, run the following:

First, return to main:

```
git checkout main
```

Next, download the latest main branch from GitHub and update your local main branch.

```
git pull origin main
```

💡 Step 11: Return to Your Branch

```
git checkout data/data-cleaning
```

💡 Step 12: Merge main into Your Branch

```
git merge main
```

This incorporates the latest updates from main into your branch.

If merge conflicts occur:

- Git will mark the conflicted files. Open them in an editor.
- Look for conflict markers (<<<<<<, =====, >>>>>>). Decide which changes to keep.
- Edit the files to resolve the conflicts, then save.

Stage the resolved files:

```
git add .
```

Complete the merge

If Git has not already created the merge commit, complete the merge:

```
git commit -m "Merge main into feature/data-cleaning"
```

Push the updated branch:

```
git push
```

💡💡💡 Tip: If you're unsure how to resolve a conflict, ask a teammate for help before merging.

Checking for Conflicts (Before Merging)

You can check differences between your branch and main without performing a merge:

```
git diff main
```

To see a compact history graph:

```
git log --oneline --graph --all
```

The most reliable conflict check is to actually run `git merge main` – Git will automatically detect and report any conflicts.

💡 Creating a Pull Request (PR)

After your branch is ready and pushed:

Open the repository on GitHub.

Once you have pushed branch, go to the repository on GitHub. You'll often see a **"Compare & pull request"** button for your recently pushed branch.

Click it.

If not,

- Go to the **"Pull requests"** tab and click **"New pull request"**.
- Set base to **main**. The base branch is where changes will be merged into (main)
- Set compare to your feature branch (example, `data/data-cleaning`). The compare branch is the branch containing your work.
- Review the changes shown.
- Add a clear title.
- Write a description explaining what changes were made and why.
- Read "Linking Issues to Work" below on how to automatically close an issue with PR
- Attach screenshots or snapshots (if necessary)
- Click **"Create pull request"**.

Example PR description:

Title: Add data cleaning module

Description:

- Added preprocessing functions
- Updated documentation
- Added unit tests
- Closes #12 (Read "Linking Issues to Work" below)

💡 Linking Issues to Work

When working on an issue:

- Create a feature branch for the task.
- Open a Pull Request (PR) when the work is ready.
- Reference the issue number in the PR description when applicable.

- When the PR is merged into main, GitHub automatically closes all three issues in the example below. Git scans the PR description and automatically closes the issue when it detects these keywords:

Example:

```
#To link to and close an issue:
Closes #12
Resolves #11
Fixes #15

#To link to an issue but not close it:
#8
Related to #5
```

💡 Reviewing and Merging a PR

Review PR

- Every PR must be reviewed by at least one team member
- The reviewer must check that code is correct, clean and safe to merge into **main**
- If any issues or concerns are found, the reviewer leaves a comment
- The author will make updates on the same branch, commit the changes, and push them to Github
- The PR is automatically updated with the new commits (a Pull Request is tied to a branch, not to a specific commit).

After the PR is reviewed and approved:

- Once approved, merge the branch into **main**

Click one of the merge options:

- **Merge Pull Request** (creates a merge commit)
- **Squash and Merge** (combines all commits into one)

Once merged, the changes become part of **main**.

If the branch is no longer needed after the merge:

- Delete the feature branch to keep the repository clean.

💡 Update Local Repository After a PR is Merged

After the PR is merged on GitHub, update your local repository:

```
git checkout main
git pull origin main
```

Your local **main** branch is now synchronized with GitHub.

💡 Deleting Branches

If the feature branch is no longer needed, you may delete it locally and remotely.

Delete local branch (safe, only if already merged):

```
git branch -d data/data-cleaning
```

Force delete the local branch (even if not merged):

```
git branch -D data/data-cleaning
```

Delete the remote branch on Github:

```
git push origin --delete data/data-cleaning
```

Recommended Team Workflow (Visual)

```
Update main
  ↓
Create branch
  ↓
Make changes → Commit → Push
                  (first push: set upstream tracking)
  ↓
Update branch with latest main (merge main)
  ↓
Create Pull Request
  ↓
Review → Merge into main
  ↓
Pull updated main locally
```

Important Rules

❌ Don't

Commit directly to **main**

Make huge, rare commits

✅ Do

Always create a branch first

Commit frequently (small commits)

✗ Don't**✓ Do**

Open a PR without updating from **main** Pull latest **main** before PR

Use vague branch names

Use descriptive names like **feature/user-login**

Typical End-to-End Workflow (Commands)

```
# Start from an updated main
git checkout main
git pull origin main

# Create and switch to a new branch
git checkout -b feature/my-feature

# Make changes in your editor, then stage and commit
git add .
git commit -m "Describe changes"

# First push: create remote branch and set upstream tracking
git push -u origin feature/my-feature

# [Optional] Continue working, commit, and push as needed
# git add .
# git commit -m "Another change"
# git push      # upstream is already set

# Before opening a PR, update your branch with latest main
git checkout main
git pull origin main
git checkout feature/my-feature
git merge main      # Resolve conflicts if any
git push

# Now create Pull Request on GitHub UI

# After PR is merged, update local main
git checkout main
git pull origin main

# Delete your feature branch locally and remotely
git branch -d feature/my-feature
git push origin --delete feature/my-feature
```

Command Summary Table

Terminal Command Summary

Command	Type	Purpose
<code>git clone <url></code>	Git	Clone a repository
<code>cd <folder></code>	Shell	Enter repository directory
<code>git branch</code>	Git	List local branches
<code>git branch -vv</code>	Git	Show branches and their upstream tracking
<code>git checkout main</code>	Git	Switch to <code>main</code> branch
<code>git checkout <branch></code>	Git	Switch to an existing branch
<code>git checkout -b <branch></code>	Git	Create and switch to new branch
<code>git pull origin main</code>	Git	Update local <code>main</code> from origin/ <code>main</code>
<code>git status</code>	Git	Show file changes
<code>git add <file></code>	Git	Stage a specific file
<code>git add .</code>	Git	Stage all changes in current directory
<code>git commit -m "message"</code>	Git	Create a commit
<code>git push -u origin <branch></code>	Git	First push (set upstream)
<code>git push</code>	Git	Push subsequent commits to the tracked remote branch
<code>git merge main</code>	Git	Merge <code>main</code> into current branch
<code>git diff main</code>	Git	Compare current branch with <code>main</code>
<code>git log --oneline --graph --all</code>	Git	View commit history as graph
<code>git branch -d <branch></code>	Git	Delete local branch (safe)
<code>git branch -D <branch></code>	Git	Force delete local branch
<code>git push origin --delete <branch></code>	Git	Delete remote branch

One-Line Cheat Sheet

```
git checkout main && git pull origin main && git checkout -b feature/my-feature && git add . && git commit -m "Describe changes" && git push -u origin feature/my-feature
```

Before PR:

```
git checkout main && git pull origin main && git checkout feature/my-feature && git merge main && git push
```

Resolve conflicts if prompted

```
# After PR merge:  
git checkout main && git pull origin main
```

Note: The one-liner above chains commands with &&. If any command fails, the chain stops – that's intentional for safety.